Undergraduate Project: Finite-Precision Phases in Quantum Signal Processing

Itay Hen

September 26, 2026

Overview

This project measures how much accuracy a quantum signal processing (QSP) circuit loses when its phase angles are stored with a finite number of digits. The student will take phase lists produced by standard open-source solvers for real target functions, round them in several ways, and compare the resulting errors with the theoretical bounds.

The work is entirely classical and numerical: Python, NumPy, and two existing phase-finding packages. The deliverable is a set of figures and a short report showing how the error grows with polynomial degree and bit precision for realistic phase lists. To our knowledge this has not yet been measured systematically on solver output.

Motivation

QSP and its matrix generalization, the quantum singular value transformation (QSVT), are now the standard way to build quantum algorithms for Hamiltonian simulation, matrix inversion, and eigenvalue filtering. A QSP circuit is specified by a polynomial and a list of $d + 1$ phase angles, where d is the polynomial degree. The phases are computed classically, written to a file, and then compiled into rotation gates.

Theory usually treats those phases as exact real numbers. In practice they pass through a solver with limited reliable digits, get printed or packed into a fixed-width binary word, and are then approximated by gates. Each step introduces error. This project isolates one of them: the error from storing the phases with finite precision.

A simple argument bounds this error. If every phase is off by at most δ, the realized circuit differs from the ideal one by at most $(d + 1)\delta$ in operator norm. At the endpoints $x = \pm 1$ the error depends only on the sum of the individual phase errors, and the circuit is equally sensitive to every phase, for any phase list. When the rounding errors share a sign, the bound is attained; when they cancel, the error can be far smaller. So linear growth is an attainable worst case, not something every rounded list exhibits, but it does mean a fixed number of digits cannot give a degree-independent guarantee. Rounding to r decimal places guarantees an error no worse than about $(d + 1)\, 10^{-r}/2$.

That is the worst case. If the rounding errors behaved like independent random numbers, they would partly cancel, and the typical error would grow like $\sqrt{d}$ rather than d. Which of these two regimes describes real phase files is the question this project addresses. There is reason to suspect the pessimistic one. Phase lists for smooth targets often contain long runs of nearly equal angles, and equal

angles round the same way, so their errors add up instead of canceling. In controlled experiments where a fixed fraction of the phases are exactly repeated, the growth exponent jumps from 1/2 to nearly 1 once only a quarter of the list repeats.

There is also a cheap remedy. Instead of rounding each phase independently, one can carry the rounding residual forward to the next phase, a technique called first-order noise shaping or sigma-delta ($\Sigma\Delta$) quantization. This keeps every partial sum of the errors within half a grid step. At the endpoints the error then no longer grows with d at all, and in the interior it depends on how smoothly the phase sensitivities vary along the sequence.

All of this has so far been tested on random or artificially structured phase lists. To our knowledge it has not been checked systematically on phase lists produced by real solvers for real targets, and that is what this project does. It is also a practical question: published resource estimates sometimes use phases stored as 16 to 24-bit words, where this error can dominate at the degrees people actually use.

Required background

No prior quantum computing course is needed. Everything the project uses is below, and the reading list in the next section fills in the details. Plan on spending the first two weeks here.

Single-qubit algebra

You should be comfortable with the Pauli matrices X and Z, with 2×2 unitary matrices, and with the operator norm $\| A \|$ (the largest singular value). The two building blocks are a Z-rotation and a "signal" rotation that depends on a number x in $[-1,1]$:

$$R_z(\phi) = e^{i\phi Z} = \begin{pmatrix} e^{i\phi} & 0 \\ 0 & e^{-i\phi} \end{pmatrix}, \qquad W(x) = e^{i\arccos(x)X} = \begin{pmatrix} x & i\sqrt{1-x^2} \\ i\sqrt{1-x^2} & x \end{pmatrix}.$$

Chebyshev polynomials

The Chebyshev polynomials satisfy $T_k(\cos\theta) = \cos(k\theta)$. They are the natural basis for QSP because $W(x)$ is a rotation by the angle $\theta = \arccos x$, so products of $W(x)$ produce trigonometric functions of $k\theta$. You need the three-term recurrence, parity (T_k is even or odd with k), and the fact that sampling at Chebyshev points $x = \cos\theta_\ell$ is the stable way to evaluate polynomials on $[-1,1]$. Chapters 1 to 4 of Trefethen's book cover this.

Quantum signal processing

Given phases $\Phi = (\phi_0, \ldots, \phi_d)$, QSP forms the product

$$U_\Phi(x) = R_z(\phi_0) \prod_{j=1}^{d} \left[W(x)\, R_z(\phi_j) \right], \qquad P_\Phi(x) = \langle 0 | U_\Phi(x) | 0 \rangle.$$

The matrix element $P_\Phi(x)$ is a polynomial in x of degree at most d, with parity $d \bmod 2$, and $|P_\Phi(x)| \leq 1$ on $[-1,1]$. Conversely, almost any polynomial with these properties can be realized by some choice of phases. Many applications use symmetric phases, $\phi_j = \phi_{d-j}$, for which the target real polynomial

appears as the real or imaginary part of P_Φ, depending on the package (QSPPACK uses the real part, pyqsp's sym_qsp method the imaginary part).

The forward direction (phases to polynomial) is just matrix multiplication, which you will code yourself in an afternoon. The inverse direction (polynomial to phases) is a hard numerical problem, and for that you will use existing solvers.

One warning: papers and packages use different conventions (W_x versus W_z, where the phases sit, whether the readout is P, $\mathrm{Re}\,P$, or $\langle+|U|+\rangle$). Before comparing anything, you must check that your forward code reproduces each solver's own output.

Qubitization and QSVT, in one paragraph

You do not need to implement these, but you should know why the scalar problem matters. Suppose a large matrix A (for example a Hamiltonian divided by a normalization) is embedded as the top-left block of a unitary. Qubitization shows that this unitary acts on each eigenvalue λ of A like $W(\lambda)$ inside its own two-dimensional subspace. QSVT then interleaves it with phase rotations to apply P_Φ to every eigenvalue (or singular value) at once. So an error in the scalar polynomial $P_\Phi(x)$ becomes an error in the matrix function $P_\Phi(A)$ of the same size. That is why the whole project can be done with 2×2 matrices.

The error bounds you will test

For perturbed phases $\Phi + \delta\Phi$, a telescoping argument gives, for every x,

$$\| U_{\Phi+\delta\Phi}(x) - U_\Phi(x) \| \leq \sum_{j=0}^{d} 2\,\left|\sin\left(\delta\phi_j/2\right)\right| \leq \sum_{j=0}^{d} \left|\delta\phi_j\right| \leq (d+1)\max_j \left|\delta\phi_j\right|.$$

At $x = 1$ the signal rotation is the identity, all factors commute, and the error is known exactly:

$$U_\Phi(1) = e^{\,iZ\sum_j \phi_j}, \qquad \| U_{\Phi+\delta\Phi}(1) - U_\Phi(1) \| = 2\left|\sin\left(1/2\sum_j \delta\,\phi_j\right)\right|.$$

If instead the errors were independent and uniform on $[-h/2, h/2]$, where h is the rounding grid step, the typical (root-mean-square) error would be about $h\sqrt{(d+1)/12}$.

Rounding schemes

Direct rounding replaces each ϕ_j by the nearest multiple of h. Noise-shaped ($\Sigma\Delta$) rounding carries the leftover error forward:

$$u_j = \phi_j + e_{j-1}, \qquad \hat{\phi}_j = h\,\mathrm{round}\left(u_j/h\right), \qquad e_j = u_j - \hat{\phi}_j, \qquad e_{-1} = 0.$$

Every partial sum of the errors then equals $-e_k$, which is at most $h/2$ in magnitude. A cruder variant, the one-phase sum correction, rounds directly and then adjusts the last phase so that the total phase is preserved on the grid. Randomized (dithered) rounding is a third option that breaks correlations by adding noise before rounding.

Reading list

Read the starred items first, in order. The rest are for reference when a specific question comes up.

Getting oriented

- ★ J. M. Martyn, Z. M. Rossi, A. K. Tan, I. L. Chuang, Grand unification of quantum algorithms, PRX Quantum 2, 040203 (2021). The best single introduction to QSP, qubitization, and QSVT, with worked examples. Read Sections I to IV closely.
- ★ L. Lin, Lecture notes on quantum algorithms for scientific computation, arXiv:2201.08309. The chapters on block encoding and QSP give a careful, math-first treatment.
- M. A. Nielsen and I. L. Chuang, *Quantum Computation and Quantum Information* (Cambridge, 2010), Chapters 2 and 4. Only if single-qubit gates and Dirac notation are new to you.

Original QSP and QSVT papers

- G. H. Low and I. L. Chuang, Optimal Hamiltonian simulation by quantum signal processing, PRL 118, 010501 (2017).
- G. H. Low and I. L. Chuang, Hamiltonian simulation by qubitization, Quantum 3, 163 (2019).
- A. Gilyén, Y. Su, G. H. Low, N. Wiebe, Quantum singular value transformation and beyond, STOC 2019. Long and technical; skim Section 3 for the structure.

Computing the phases

- ★ Y. Dong, X. Meng, K. B. Whaley, L. Lin, Efficient phase-factor evaluation in quantum signal processing, PRA 103, 042419 (2021). The symmetric-QSP optimization method you will use most.
- R. Chao, D. Ding, A. Gilyén, C. Huang, M. Szegedy, Finding angles for quantum signal processing with machine precision, arXiv:2003.02831. The basis of pyqsp's laurent method.
- J. Wang, Y. Dong, L. Lin, On the energy landscape of symmetric quantum signal processing, Quantum 6, 850 (2022). Explains why the optimization converges.
- L. Lin, Mathematical and numerical analysis of quantum signal processing, arXiv:2510.00443 (2025). A recent survey including floating-point stability.

Phase errors and precision

- ★ The group's draft manuscript, "Finite-Precision Phase Synthesis in QSP and QSVT" (available from the supervisor). Sections 4, 6, 9, and 11 are the ones this project tests.
- A. K. Tan, Y. Liu, M. C. Tran, I. L. Chuang, Perturbative model of noisy quantum signal processing, PRA 107, 042429 (2023). Phase errors treated as hardware noise.
- M. Steudtner et al., Fault-tolerant quantum computation of molecular observables, Quantum 7, 1164 (2023). A resource estimate that stores QSP angles as 21-bit words, which is the regime where this project matters.
- N. J. Ross and P. Selinger, Optimal ancilla-free Clifford+T approximation of z-rotations, QIC 16, 901 (2016). How rotation angles become gate counts; background only.

Numerical background

- ★ L. N. Trefethen, *Approximation Theory and Approximation Practice*, extended edition (SIAM, 2019), Chapters 1 to 4 and 8. Chebyshev polynomials and interpolation.

- N. J. Higham, *Accuracy and Stability of Numerical Algorithms*, 2nd ed. (SIAM, 2002), Chapters 1 and 2. Floating-point error and how to reason about it.
- For noise shaping, any introduction to delta-sigma modulation will do; S. Pavan, R. Schreier, G. C. Temes, *Understanding Delta-Sigma Data Converters*, 2nd ed. (Wiley-IEEE, 2017), Chapter 1, is a standard one. I. Daubechies and R. DeVore, Approximating a bandlimited function using very coarsely quantized data, Annals of Math. 158, 679 (2003), is the mathematical treatment, for the stretch goal only.

Software

- pyqsp (Python). Use the sym_qsp method; it has built-in targets for Hamiltonian simulation (hamsim), inversion (invert), sign and threshold functions, and eigenstate filters.
- QSPPACK (MATLAB). The reference implementation of the Dong et al. solver; useful as a cross-check.
- NumPy's `numpy.polynomial.chebyshev` module for Chebyshev coefficients and evaluation, and mpmath if you need extended-precision reference values.

Project description

The project runs in five stages, with a firm checkpoint after Stage 2. The core assignment carries all five stages through on two target families at modest scale. Everything marked as an extension waits until that core result is reproducible.

Stage 1: forward QSP code and convention check

Write a short Python module that, given a phase list and an array of x values, returns $U_\Phi(x)$ and $P_\Phi(x)$. Vectorize over x with NumPy; a loop over the $d+1$ phases is fine.

Validate it three ways:

1. Reproduce pyqsp's own response curves for two or three of its built-in examples, in the package's own convention.
2. Confirm numerically that $U_\Phi(1) = \exp\left(iZ\sum_j \phi_j\right)$ for random phase lists.
3. Confirm that $|P_\Phi(x)| \le 1$ and that the parity matches d.

Output: a tested module and a one-page note stating exactly which convention you use and how it maps to pyqsp and QSPPACK.

Stage 2: a library of real phase lists

Generate phase lists with a production solver, using pyqsp's sym_qsp method as the default and QSPPACK as a cross-check. The core assignment uses the first two target families below; the other two are extensions.

Target	Used for	Parameter to vary	Scope
cos(τx) and sin(τx), Jacobi–Anger	Hamiltonian simulation	evolution time τ	Core
Sign or step function, scaled	amplitude amplification, eigenvalue	sharpness (inverse	Core

Target	Used for	Parameter to vary	Scope
to max 0.9	thresholding	gap)	
Scaled inverse 1/(2κx) on 1/κ ≤ |x| ≤ 1	matrix inversion	condition number κ	Extension
Eigenstate or bandpass filter	ground-state preparation	filter width	Extension

Every target must be scaled so that $|f(x)| \leq 1$ wherever it is approximated, because a QSP matrix element cannot exceed one in magnitude. The inverse is the case to watch: $1/x$ reaches κ at $|x| = 1/\kappa$, so the factor $1/(2\kappa)$ brings its maximum down to 1/2. Its approximation region is $1/\kappa \leq |x| \leq 1$; on $|x| < 1/\kappa$ the polynomial is constrained only by $|p| \leq 1$.

For the core families, cover degrees from about 10 to about 500, spaced roughly logarithmically. Pushing to 2,000 is an extension. Save each list at full double precision together with its solver residual (the distance between the realized polynomial and the target). This residual is a different error from the one we study, and it should be recorded but not mixed in.

Also plot a few phase lists, ϕ_j against j. Note whether they contain long runs of nearly equal values, since that structure is what the theory says controls the answer.

Output: a directory of phase files with metadata (target, parameters, degree, solver, residual).

Checkpoint

Before starting Stage 3, review three things with the supervisor: the Stage 1 convention note, the reproduction of each solver's own output, and the core phase library with its residuals and plots of ϕ_j against j. Move on only when all of these regenerate cleanly from the repository.

Stage 3: quantize

Round every core phase list on a grid of step h. The primary grid is a b-bit word spanning 2π ($h = 2\pi/2^b$), with b = 12, 16, 20, 24, 28, and 32. Use three schemes:

1. Direct nearest rounding.
2. Direct rounding plus the one-phase sum correction.
3. First-order $\Sigma\Delta$ rounding.

Decimal grids and dithered rounding (uniform noise on $[-h/2, h/2]$ added before rounding, over a handful of seeds) are extensions. Record the error sequence $\delta\phi_j = \hat{\phi}_j - \phi_j$ for each case.

Output: rounded phase files, one per target, degree, grid, and scheme.

Stage 4: measure the realized error

For every rounded list, compute the following on a Chebyshev grid of at least $10(d+1)$ points in $[-1,1]$:

1. The full-unitary error, $\max_x \| U_{\hat{\Phi}}(x) - U_{\Phi}(x) \|$.
2. The matrix-element error, $\max_x |P_{\hat{\Phi}}(x) - P_{\Phi}(x)|$.
3. The readout error in whichever part (real or imaginary) carries the target.

4. The same three quantities restricted to the region the application uses, for example $|x| \geq 1/\kappa$ for inversion.

Keep these separate throughout. They can behave differently, especially near $x = \pm 1$, where the readout error can vanish to first order even when the unitary error is large.

Also compute the normalized autocorrelation of each $\delta\phi$ sequence. For a subset of cases, compute the sensitivity profile $g_j(x) = \partial P_\Phi(x)/\partial\phi_j$ using prefix and suffix products, and its total variation along j.

Spot-check a few high-degree cases with mpmath at 30 or more digits, to make sure double-precision evaluation is not what limits the smallest errors.

Output: one table of results (CSV) covering every case.

Stage 5: analysis

Use the results table to answer five questions, each with one or two figures:

1. **Scaling with degree.** At fixed h, how does the error grow with d for each target and scheme? Fit the exponent on a log-log plot and compare with 1 (coherent, $(d+1)h/2$) and 1/2 (independent, $h\sqrt{(d+1)/12}$).
2. **What predicts the exponent.** Does the autocorrelation of $\delta\phi$, or the presence of plateaus in the phase list, predict where each target lands between those two limits?
3. **Where the error lives.** Is the maximum error at the endpoints or in the interior? Does this change between the unitary metric and the readout metric?
4. **How much noise shaping helps on real lists.** Compare the schemes at equal h. Check whether the interior gain from $\Sigma\Delta$ tracks the total variation of the sensitivity profile, as the theory predicts.
5. **Bits needed.** For target accuracies of 10^{-4}, 10^{-6}, and 10^{-8}, plot the number of bits needed against d for each scheme. As a concrete check, find the degree at which 21-bit phases stop meeting 10^{-4} for each target.

Stretch goal: does preserving symmetry help in the interior?

First-order $\Sigma\Delta$ runs left to right, so a palindromic list $\phi_j = \phi_{d-j}$ generally comes out non-palindromic. A symmetric version is easy to build: apply $\Sigma\Delta$ to the first half and mirror it. When the list has an even number of phases, the partial sums of the error then stay within $3h/2$. When it has an odd number, rounding the center phase on its own gives $2h$. Deriving these two bounds is a good warm-up exercise. The endpoint guarantee therefore survives symmetrization at a small constant cost.

The open question is what happens away from the endpoints, in the readout the application actually uses. Symmetric phases keep the target in one part (real or imaginary) of P_Φ, while plain $\Sigma\Delta$ may leak error into the other part. On the core library, compare the interior readout error of direct rounding, plain $\Sigma\Delta$, and mirrored $\Sigma\Delta$. Then check whether the differences track the total variation of the sensitivity profile. If time remains, try a mirrored second-order recursion and see whether it gains anything at the degrees studied.

Prerequisites, timeline, and deliverables

Prerequisites. Linear algebra at the level of eigenvalues, unitary matrices, and matrix norms; comfort with complex numbers; and working Python with NumPy and Matplotlib. A numerical methods course helps but is not required. No quantum computing background is assumed.

Timeline. The plan below assumes a 15-week semester at roughly 10 hours per week.

Weeks	Work
1–2	Starred readings; Stage 1 code and convention note
3–5	Stage 2 core phase library; checkpoint at end of week 5
6–7	Stage 3 quantization
8–10	Stage 4 error measurements
11–13	Stage 5 analysis and figures
14–15	Written report; extensions and stretch goal if time allows

Deliverables.

- A code repository with a README that regenerates every figure from scratch.
- The phase library and the results table (CSV).
- A report of 8 to 12 pages covering the setup, the five Stage 5 questions, and what remains open.

What counts as success. The core result is Stages 1 to 5 for the two core families, with a reproducible answer to the first Stage 5 question: do real phase lists round coherently or not? Extensions add the other two families, degrees up to 2,000, decimal grids, dithering, and the stretch goal. Whether the results are new, and whether they belong in a paper, is decided after the measurements are in hand.